

Projeto Prático React + JSON Server

Oscar Archive

Objetivo Geral

Desenvolver uma aplicação React que permita consultar, cadastrar e favoritar filmes indicados ao Oscar.

O projeto foi planejado para praticar os principais conceitos estudados durante a disciplina:

- React Router DOM
- useState
- useEffect
- useContext
- useParams
- Props
- Eventos de formulário
- Renderização de listas
- Renderização condicional
- Bootstrap
- React Bootstrap
- Consumo de APIs
- JSON Server

Arquitetura do Projeto

A aplicação será dividida em dois projetos independentes.

Projeto 1 — API

Responsável pelo armazenamento dos dados.

Utilizará:

- Node.js
- JSON Server

Estrutura:

oscar-api/

Projeto 2 — Frontend

Responsável pela interface da aplicação.

Utilizará:

- React
- Vite
- React Router DOM
- Bootstrap
- React Bootstrap

Estrutura:

```
oscar-react/
```

Comunicação Entre os Projetos

Os dois projetos serão executados separadamente.

Fluxo:

```
graph TD
    A[Frontend React] --> B[Requisições HTTP]
    B --> C[JSON Server]
```

O React consumirá os dados disponibilizados pela API Fake criada com JSON Server.

Esta separação tem como objetivo simular uma arquitetura real de aplicações web modernas.

Objetivos de Aprendizagem

Ao concluir o projeto, o aluno deverá ser capaz de:

- Criar uma API Fake utilizando JSON Server.
- Consumir dados externos utilizando fetch.
- Organizar aplicações React em múltiplas páginas.
- Trabalhar com Context API.
- Compartilhar estados globais.

- Utilizar parâmetros de rota.
 - Trabalhar com formulários controlados.
 - Implementar favoritos.
 - Utilizar Bootstrap para estilização.
 - Organizar uma aplicação em componentes reutilizáveis.
-

Tema da Aplicação

Oscar Archive

A aplicação exibirá filmes indicados ao Oscar.

Importante:

O sistema NÃO armazenará apenas vencedores.

Serão armazenados:

- Filmes vencedores.
- Filmes apenas indicados.

Cada filme possuirá um atributo que indicará se ele venceu ou não.

Campo:

vencedor

Valores possíveis:

true
false

Estrutura de Dados

Filme

Cada filme deverá possuir os seguintes dados:

- id
- titulo
- ano
- diretor
- categoria

- sinopse
 - imagem
 - vencedor
-

Usuário

Cada usuário deverá possuir:

- id
- nome
- email
- senha
- favoritos

O campo favoritos armazenará uma lista de IDs de filmes.

Context API

A aplicação utilizará Context API como mecanismo principal de gerenciamento de estado global.

FilmeContext

Responsável por:

- Carregar filmes da API.
 - Buscar filme por ID.
 - Listar vencedores.
 - Listar indicados.
 - Adicionar filme.
 - Atualizar filme.
 - Remover filme.
-

UsuarioContext

Responsável por:

- Login.
 - Logout.
 - Usuário logado.
 - Favoritos.
 - Cadastro de usuário.
-

AppProvider

Responsável por agrupar todos os Providers da aplicação.

Estrutura conceitual:

```
AppProvider
├── FilmeProvider
└── UsuarioProvider
```

Utilização no main.jsx

O AppProvider será utilizado no ponto de entrada da aplicação.

Objetivo:

Disponibilizar todos os contextos para qualquer componente da aplicação.

Essa estratégia já foi utilizada em sala e deverá ser mantida neste projeto.

Estrutura de Pastas do Frontend

```
src
├── assets
├── components
├── contexts
│   ├── FilmeContext
│   ├── UsuarioContext
│   └── AppProvider
├── pages
├── routes
├── services
└── App
```

Explicação das Pastas

assets

Armazenamento de imagens e recursos estáticos.

components

Componentes reutilizáveis da aplicação.

Exemplos:

- Header
 - Footer
 - MovieCard
 - Banner
 - FavoriteButton
-

contexts

Armazenamento dos Context Providers.

pages

Páginas da aplicação.

Cada rota deverá possuir sua própria página.

routes

Configuração das rotas da aplicação.

services

Centralização das chamadas à API.

Páginas da Aplicação

Home

Objetivo:

Apresentar os destaques da plataforma.

Exibir:

- Filmes vencedores recentes.
- Filmes indicados.
- Filmes em destaque.

Conceitos:

- useEffect
 - Renderização de listas
-

Lista de Filmes

Rota:

/filmes

Objetivo:

Exibir todos os filmes cadastrados.

Conceitos:

- useEffect
 - Props
 - Renderização de listas
-

Detalhes do Filme

Rota:

/filmes/:id

Objetivo:

Exibir todas as informações do filme.

Conceitos:

- useParams
 - useEffect
-

Cadastro de Filme

Rota:

/cadastro-filme

Objetivo:

Cadastrar novos filmes.

Conceitos:

- useState
 - Eventos de formulário
-

Cadastro de Usuário

Rota:

/cadastro-usuario

Objetivo:

Cadastrar usuários.

Conceitos:

- useState
 - Formulários
-

Login

Rota:

/login

Objetivo:

Realizar login do usuário.

Conceitos:

- useState
- Context API

Favoritos

Rota:

/favoritos

Objetivo:

Exibir filmes favoritados pelo usuário.

Conceitos:

- Context API
- Renderização de listas

Componentes Reutilizáveis

A aplicação deverá possuir componentes reutilizáveis.

Sugestões:

- Header
- Footer
- MovieCard
- MovieList
- FavoriteButton
- WinnerBadge
- SearchBar
- Banner

Renderização Condicional

A aplicação deverá utilizar renderização condicional em diferentes cenários.

Exemplos:

Filme vencedor

Exibir:

 Vencedor do Oscar

Filme indicado

Exibir:

 Indicado ao Oscar

Usuário logado

Exibir:

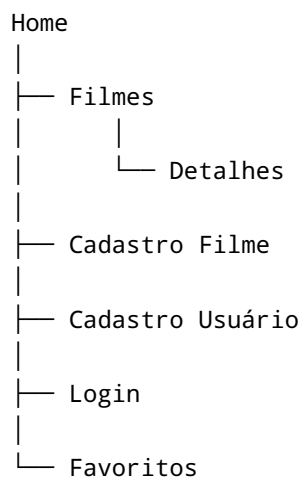
- Favoritos
 - Logout
-

Usuário não logado

Exibir:

- Login
 - Cadastro
-

Fluxo de Navegação



Estrutura Inicial da API

A API possuirá inicialmente duas coleções:

filmes

Armazenará todos os filmes indicados ao Oscar.

usuarios

Armazenará os usuários da plataforma.

Próxima Etapa

Na próxima etapa será construída a API inicial contendo:

- Base de filmes indicados ao Oscar.
- Identificação dos vencedores.
- Usuários cadastrados.
- Lista de favoritos vinculada aos usuários.
- Estrutura completa do arquivo db.json para utilização com JSON Server.